

10 Profiling

Content

10.1 The Command time	10-3
10.2 The Command strace	10-5
10.3 gprof	10-6
10.4 gcov (Coverage Testing)	10-11

After all bugs have been found and fixed (using debugging tools), we might want to **tune the performance** of the program.

Profiling tools are used to analyze the run time behaviour of a program, for example:

- **How often** are certain functions / code lines executed?
- **How much time** is spent in certain functions / code lines?

Afterwards we know in which parts it might be worth doing additional optimization.

(The example programs will be in C++, Fortran versions are available in **Profiling/Fortran**.)

Example: Computation of all prime numbers up to some limit n

In our first approach we test each candidate

- if it is even or
- if any odd number d , $3 \leq d \leq \sqrt{n}$, is a divisor.

Profiling/C++/primes1.cc

```
13 bool isprime(int i)
14 {
15     if (i % 2 == 0)
16         return false;
17     for (int d{3}; d <= sqrt(static_cast<double>(i)); d += 2)
18         if (i % d == 0)
19             return false;
20     return true;
21 }
22
23 int main(int argc, char** argv)
24 {
25     int n{1000}; // default number
26     string outfilename{"output.txt"}; // default output file name
27     ofstream outfile;
28     if (argc >= 2)
29         n = atoi(argv[1]);
30     if (argc >= 3)
31         outfilename = argv[2];
32     outfile.open(outfilename);
33     if (n >= 2)
34         outfile << 2 << endl;
35
36     for (int i{3}; i <= n; i += 2)
37         if (isprime(i))
38             outfile << i << endl;
39
40     outfile.close();
41 }
```

10.1 The Command time

For a first overview, a program can be run via the command `time`:

- In this and all following measurements we determine all 1 857 859 prime numbers between 2 and 30 000 000.

```
.../Profiling/C++ > make primes1  
g++ -std=c++23 -Wall -c primes1.cc  
g++ primes1.o -o primes1  
.../Profiling/C++ > time primes1 30000000
```

```
real    0m6.267s  
user    0m6.017s  
sys     0m0.247s
```

- meaning of the three output lines:
 - real: elapsed time (so-called wall-time)
 - user: CPU time that the process spent in user mode
 - sys: CPU time that the process spent in kernel mode

- Most of the system time is spent for the output of the prime numbers into a file.

As we do not want to consider this part for optimization, we can write the results into the “wastebasket file” `/dev/null`.

```
.../Profiling/C++ > time primes1 30000000 /dev/null
```

```
real    0m6.145s
user    0m6.090s
sys     0m0.053s
```

- The `time` command is well suited for testing the impact of different compiler optimization flags:

```
.../Profiling/C++ > make primes1_03
g++ -std=c++23 -Wall -O3 -c primes1.cc -o primes1_03.o
g++ primes1_03.o -o primes1_03
.../Profiling/C++ > time primes1_03 30000000 /dev/null
```

```
real    0m5.219s
user    0m5.145s
sys     0m0.073s
```

10.2 The Command strace

The command `strace -c` prints a summary what system calls are comprised in the system time.

```
.../Profiling/C++ > strace -c primes1 30000000
% time      seconds  usecs/call   calls   errors syscall
-----
 99.57      0.674073         0 1857859         0 write
  0.31      0.002077         57    36         30 openat
  0.12      0.000817        136     6         0 close
  0.00      0.000000         0     4         0 read
  0.00      0.000000         0     5         0 fstat
  0.00      0.000000         0    22         0 mmap
  0.00      0.000000         0     6         0 mprotect
  0.00      0.000000         0     1         0 munmap
  0.00      0.000000         0     3         0 brk
  0.00      0.000000         0     2         0 pread64
  0.00      0.000000         0     1         1 access
  0.00      0.000000         0     1         0 execve
  0.00      0.000000         0     1         0 arch_prctl
  0.00      0.000000         0     1         0 futex
  0.00      0.000000         0     1         0 set_tid_address
  0.00      0.000000         0     9         6 newfstatat
  0.00      0.000000         0     1         0 set_robust_list
  0.00      0.000000         0     1         0 prlimit64
  0.00      0.000000         0     1         0 getrandom
  0.00      0.000000         0     1         0 rseq
-----
100.00      0.676967         0 1857962        37 total
```

10.3 gprof

In the next step, we analyze the runtime behaviour with a profiler.

URL: GNU gprof is part of the binutils package: <https://www.gnu.org/software/binutils/>

documentation: <https://sourceware.org/binutils/docs/gprof/>

- As a preparation for profiling, the program has to be compiled and linked with `-pg`.

```
.../Profiling/C++ > make primes1_pg  
g++ -std=c++23 -Wall -pg -g -c primes1.cc -o primes1_pg.o  
g++ -pg primes1_pg.o -o primes1_pg
```

- While this program is executed, profiling data is collected in the file `gmon.out`.

```
.../Profiling/C++ > primes1_pg 30000000 /dev/null  
.../Profiling/C++ > ls -l gmon.out  
-rw-r--r-- 1 holger users 7800 Feb 19 16:49 gmon.out
```

- The command `gprof primes1_pg` generates two different profiles which will be explained separately in the following.

Flat Profile

- The flat profile details for each function
 - what percentage of the overall time was spent for the function,
 - how many seconds were spent in a function—including and excluding calls to sub-functions,
 - the number of calls,
 - the average time per call.

```
.../Profiling/C++ > gprof --flat-profile primes1_pg
Flat profile:
```

Each sample counts as 0.01 seconds.

% time	cumulative seconds	self seconds	calls	self ns/call	total ns/call	name
83.24	4.27	4.27	14999999	284.67	284.67	isprime(int)
16.76	5.13	0.86				_init
0.00	5.13	0.00	8	0.00	0.00	std::__cxx11::basic_string<char, std::char
...						

Not surprisingly, most time was spent in `isprime`.

Call Graph

- In the call graph, there is one section for each function:
 - In each section, one function f is marked with an index number.
 - Above f , there is a list of functions that call f .
 - Below f , there is a list of functions that are called by f .

```
.../Profiling/C++ > gprof --graph primes1_pg
                        Call graph (explanation follows)
```

granularity: each sample hit covers 4 byte(s) for 0.19% of 5.13 seconds

index	% time	self	children	called	name
					<spontaneous>
[1]	83.2	0.00	4.27		main [1]
		4.27	0.00	14999999/14999999	isprime(int) [2]
		0.00	0.00	1/1	std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char>> [1]
		0.00	0.00	1/1	std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char>> [2]
		0.00	0.00	1/1	std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char>> [3]

[2]	83.2	4.27	0.00	14999999/14999999	main [1]
		4.27	0.00	14999999	isprime(int) [2]

...					

- The profile data that we have seen so far only contains our own functions.

To include profiling information for standard C++ functions, we must link special variants of the standard C library `libc` and math library `libm`, namely `libc_p` and `libm_p`.

With the current version of GCC, linking `libc_p` seems not possible, therefore we only use `libm_p`.

```
.../Profiling/C++ > make primes1_pp
g++ -std=c++23 -Wall -pg -g -c primes1.cc -o primes1_pp.o
g++ -pg -static primes1_pp.o -o primes1_pp -lm_p
```

- We can now see that the function `sqrt` is much more often invoked than necessary—4 160 092 556 times.

```
''' % cumulative self self total
time seconds seconds calls s/call s/call name
24.86 1.71 1.71 14999999 0.00 0.00 isprime(int)
24.57 3.40 1.69 _init_misc
21.95 4.91 1.51 profil
14.68 5.92 1.01 4160092556 0.00 0.00 __ieee754_sqrt

'''
index % time self children called name
[1] 39.5 0.00 2.72 1/1 _libc_start_call_main [3]
1.71 2.72 1 1 main [1]
1.71 1.01 14999999/14999999 isprime(int) [2]

'''
-----
[2] 39.5 1.71 1.01 14999999/14999999 main [1]
1.71 1.01 14999999 isprime(int) [2]
1.01 0.00 4160092556/4160092556 __ieee754_sqrt [6]
0.00 0.00 4160092556/4160092556 sqrtf64 [14]

-----
'''
[6] 14.7 1.01 0.00 4160092556/4160092556 isprime(int) [2]
1.01 0.00 4160092556 __ieee754_sqrt [6]

-----
'''
```

- As a first improvement, the square root is only computed once for each candidate.

Profiling/C++/primes2.cc

```
15  if (i % 2 == 0)
16      return false;
17  int dmax{static_cast<int>(sqrt(static_cast<double>(i)))};
18  for (int d{3}; d <= dmax; d += 2)
19      if (i % d == 0)
20          return false;
21  return true;
```

- This small change reduces the run time by more than 13 %.

```
.../Profiling/C++ > time primes2 30000000 /dev/null
```

```
real    0m5.308s
user    0m5.235s
sys     0m0.072s
```

```
.../Profiling/C++ > time primes2_03 30000000 /dev/null
```

```
real    0m5.097s
user    0m5.041s
sys     0m0.055s
```

10.4 gcov (Coverage Testing)

availability: **gcov** is part of the GNU Compiler Collection (GCC).

- More detailed profiling information can be obtained from a **statement coverage test**: It is counted how often each statement (program line) is executed.
- The program has to be compiled with two additional options; during compilation a file with suffix `.gcno` is written which contains information about the **control flow graph**.

```
.../Profiling/C++ > make primes2_cov
g++ -std=c++23 -Wall -fprofile-arcs -ftest-coverage -c primes2.cc -o primes2_cov.o
g++ -fprofile-arcs primes2_cov.o -o primes2_cov -lm
.../Profiling/C++ > ls -l primes2*.gcno
-rw-r--r-- 1 holger users 33034 Feb 19 16:44 primes2_cov.gcno
```

- On execution, run time information is collected in a data file (`*.gcda`).

```
.../Profiling/C++ > primes2_cov 30000000 /dev/null
.../Profiling/C++ > ls -l primes2*.gcda
-rw-r--r-- 1 holger users 2492 Feb 19 17:02 primes2_cov.gcda
```

- If the program is run a second time, the execution counters in the data file are not overwritten but summed up—delete the file before starting a new measurement.

- The program gcov transforms these run time data into a readable format; for each source file, an output file with suffix .gcov is created.

```
.../Profiling/C++ > gcov primes2_cov
```

```
File 'primes2.cc'  
Lines executed:95.83% of 24  
Creating 'primes2.cc.gcov'
```

```
.../Profiling/C++ > ls -l primes2*.gcov  
-rw-r--r-- 1 holger users 1671 Feb 20 11:00 primes2.cc.gcov
```

- The output file contains exact information, how often each line was executed; ##### means that the line was never executed.

```

.../Profiling/C++ > cat primes2.cc.gcov
-: 0:Source:primes2.cc
-: 0:Graph:primes2_cov.gcno
-: 0:Data:primes2_cov.gcda
-: 0:Runs:1
-: 1:// finding all prime numbers up to some n, version 2

""
14999999: 13:bool isprime(int i)
-: 14:{
14999999: 15:  if (i % 2 == 0)
#####: 16:    return false;
14999999: 17:  int dmax{static_cast<int>(sqrt(static_cast<double>(i)))};
4160092556: 18:  for (int d{3}; d <= dmax; d += 2)
4158234698: 19:    if (i % d == 0)
13142141: 20:      return false;
1857858: 21:  return true;
-: 22:}

""
15000000: 37:  for (int i{3}; i <= n; i += 2)
14999999: 38:    if (isprime(i))
1857858: 39:      outfile << i << endl;
...

```

- `gcov -b` generates additional information about the branches (**edge coverage test**).

```
.../Profiling/C++ > gcov -b primes2_cov
```

```
File 'primes2.cc'
Lines executed:95.83% of 24
Branches executed:100.00% of 34
Taken at least once:61.76% of 34
Calls executed:85.71% of 14
Creating 'primes2.cc.gcov'
```

```
.../Profiling/C++ > cat primes2.cc.gcov
```

```
14999999: 13:bool isprime(int i)
-: 14:{
14999999: 15:  if (i % 2 == 0)
branch 0 taken 0% (fallthrough)
branch 1 taken 100%
#####: 16:  return false;
14999999: 17:  int dmax{static_cast<int>(sqrt(static_cast<double>(i)))};
4160092556: 18:  for (int d{3}; d <= dmax; d += 2)
branch 0 taken 100%
branch 1 taken 1% (fallthrough)
4158234698: 19:  if (i % d == 0)
branch 0 taken 1% (fallthrough)
branch 1 taken 100%
13142141: 20:  return false;
1857858: 21:  return true;
-: 22:}
...
```

- As we can see, the test for even numbers is needless because the main program only passes odd numbers to the function.

- With **lcov** you can use the following commands to get HTML output in `html/index.html`:

```
.../Profiling/C++ > lcov -d `pwd` -c -o gcov.info  
.../Profiling/C++ > genhtml -o html gcov.info  
...
```

If multiple versions of the compiler are installed, an additional option might be needed for the first command:

```
.../Profiling/C++ > lcov --gcov-tool gcov-16 -d `pwd` -c -o gcov.info  
...
```

- As an alternative, **kcachegrind** can be used to get a graphical profiling output:

```
.../Profiling/C++ > valgrind --tool=callgrind primes1_pg 30000000 /dev/null  
==26773== Callgrind, a call-graph generating cache profiler  
.../Profiling/C++ > kcachegrind callgrind.out.26773 &
```

- improvements in version 3:
 - no more tests for even numbers
 - test is integrated into main program
 - Do not test small non-primes (9, 15, 21) as factor candidates.

Profiling/C++/primes3.cc

```

23  if (n >= 2)
24      outfile << 2 << endl;
37  if (n >= 19)
38      outfile << 19 << endl;
39
40  int dmax, d, i;
41  for (i = 23; i <= n; i += 2)
42  {
43      if (i % 3 == 0 || i % 5 == 0 || i % 7 == 0 || i % 11 == 0 ||
44          i % 13 == 0 || i % 17 == 0 || i % 19 == 0)
45          continue;
46      dmax = static_cast<int>(sqrt(static_cast<double>(i)));
47      for (d = 23; d <= dmax; d += 2)
48          if (i % d == 0)
49              break;
50      if (d > dmax)
51          outfile << i << endl;
52  }

```

- result:

```
.../Profiling/C++ > time primes3 30000000 /dev/null
```

```
real    0m5,194s
user    0m5,124s
sys     0m0,057s
```

```
.../Profiling/C++ > time primes3_03 30000000 /dev/null
```

```
real    0m5,063s
user    0m4,992s
sys     0m0,063s
```

Further Improvements (or: What the Profiler is not Able to Find)

- In our example, significantly better run times can be achieved by replacing the whole algorithm by **Eratosthenes' sieve**.
- The system time can be further reduced by changing “<< endl” to “<< '\n'”, i. e. buffered output.

Profiling/C++/eratosthenes.cc

```

24  int* lst{new int[n + 1]};
25  for (int j{2}; j <= n; ++j)
26      lst[j] = j;
27  int sqrtn{static_cast<int>(sqrt(static_cast<double>(n)))};
28  for (int i{2}; i <= sqrtn; ++i)
29      if (lst[i] != 0)
30          for (int j{i * i}; j <= n; j += i)
31              lst[j] = 0;
32
33  for (int j{2}; j <= n; ++j)
34      if (lst[j] != 0)
35          outfile << j << '\n';
36  delete[] lst;

```

- result:

```
.../Profiling/C++ > time eratosthenes 30000000 /dev/null
```

```
real    0m0,297s
user    0m0,289s
sys     0m0,008s
```

```
.../Profiling/C++ > time eratosthenes_03 30000000 /dev/null
```

```
real    0m0,211s
user    0m0,201s
sys     0m0,011s
```